
TASK S3.01

Table Manipulation

Level 1

- Exercise 1

La teva tasca és dissenyar i crear una taula anomenada "credit_card" que emmagatzemi detalls crucials sobre les targetes de crèdit. La nova taula ha de ser capaç d'identificar de manera única cada targeta i establir una relació adequada amb les altres dues taules ("transaction" i "company").

Challenges and solution overview

a.- Table creation

The credit_card table is created with id as the primary key to ensure unique identification of each credit card.

b.- Data types for sensitive fields

The pin and cvv fields are defined as CHAR instead of numeric types. These fields are not used for calculations, and using CHAR prevents the loss of leading zeros.

The cvv field is defined with four characters to support AMEX cards, which may use four-digit CVV values.

```

16 ● CREATE TABLE IF NOT EXISTS credit_card (
17     id VARCHAR(15) PRIMARY KEY,
18     iban VARCHAR(50),
19     pan VARCHAR(30),
20     pin CHAR(4),
21     cvv CHAR(4), -- reservo 4 bytes para futuro uso con AMEX
22     expiring_date VARCHAR(10)
23 );
24

```

100%	3:23			
Action Output				
	Time	Action	Response	Duration / Fe
✓ 1	18:18:08	CREATE TABLE I...	0 row(s) affected	0.040 sec

After creating the table 'credit_card' we import the data from the file "datos_introducir_sprint3_credit".

c.- Data import and 'date' normalization

From my point of view in these exercises we must maintain and follow the good practice of the analyst whenever possible. For this reason in this case even if the exercise does not request it and also in Level 3 we will have to revert, here I have decided to standardize the field "expiring_date".

The expiration date provided in the dataset is not in standard SQL date format. Therefore, the expiring_date column is initially defined as VARCHAR(10) and later converted to a DATE type using STR_TO_DATE.

The original format is (MM/DD/YY), which was validated by detecting the leap year 2028 (02/29/28).

To ensure data integrity, an intermediate column (expiring_date_temp) will be used during the conversion process and then renamed as the final expiring_date column.

First I present a screenshot of the entire transformation process, and in the next page I show the process step by step with its executions.

```
21  -- Fix problem with 'expiring_date' column
22  -- New Column creation
23  -- Copy values to new column formating to DATE. alter
24  -- I use WHERE with 'expiring_date' but to avoid
25  -- MyWorkBench we use de clause LIMIT
26
27  • ALTER TABLE credit_card
28    ADD expiring_date_temp DATE
29    ;
30
31  • UPDATE credit_card
32    SET expiring_date_temp = STR_TO_DATE(expiring_date, '%m/%d/%Y')
33    WHERE expiring_date IS NOT NULL
34    LIMIT 100000
35    ;
36
37  -- Data checking in order to erase original column and rename the new one
38  • SELECT expiring_date, expiring_date_temp
39    FROM credit_card
40    LIMIT 10;
41
42  -- Drop the column and rename
43  • ALTER TABLE credit_card
44    DROP COLUMN expiring_date
45    ;
46  • ALTER TABLE credit_card
47    RENAME COLUMN expiring_date_temp TO expiring_date
48    ;
```

As I have commented before, this whole part is an Off Topic that does not ask for the statement, but as we are here to learn I have decided to proceed with the transformation.

Step by Step

New temporary column creation on credit_card table

```

21  -- Fix problem with 'expiring_date' column
22  -- New Column creation
23  -- Copy values to new column formating to DATE. alter
24  -- I use WHERE with 'expiring_date' but to avoid
25  -- MyWorkBench we use de clause LIMIT
26
27  •  ALTER TABLE credit_card
28      ADD expiring_date_temp DATE
29      ;

```

100% 17:16

Action Output

	Time	Action	Response
✓ 1	23:11:29	ALTER TABLE credit_card ADD expiring_date_temp DATE	0 row(s) affected

Use of the Function `STR_TO_DATE` to convert the non standardized date to an standardized one!

To proceed is necessary to use the clause `LIMIT` to bypass the MySQL protection for massive changes. The good practice manual insist to avoid using `SET SQL_SAFE_UPDATES = 0` as a quick fix to bypass security restrictions.

```

31  •  UPDATE credit_card
32      SET expiring_date_temp = STR_TO_DATE(expiring_date, '%m/%d/%Y')
33      WHERE expiring_date IS NOT NULL
34      LIMIT 100000
35      ;

```

100% 24:27

Action Output

	Time	Action	Response
✓ 1	22:55:49	UPDATE credit_card SET expiring_date_temp = STR_TO_DATE(expiring_date, '...'	5000 row(s) affected

Check that the procedure has worked properly

```

44  -- Data checking in order to erase original column and rename the new one
45  • SELECT expiring_date, expiring_date_temp
46      FROM credit_card
47      LIMIT 10;
48

```

100% 74:44

Result Grid Filter Rows: Search Export:

expiring_date	expiring_date_temp
09/27/25	2025-09-27
12/28/28	2028-12-28
11/26/26	2026-11-26
07/27/27	2027-07-27
04/25/26	2026-04-25
11/27/26	2026-11-27
12/27/29	2029-12-27
02/28/26	2026-02-28
11/25/28	2028-11-25
02/28/25	2025-02-28

credit_card 1

Action Output

	Time	Action	Response	Duration / Fetch T
✓ 4	18:58:58	SELECT expiring_date, expiring_date_temp FRO...	10 row(s) returned	0.011 sec / 0.0006

Final process, we remove the initial column and rename the transformed column.

```

49  -- Drop the column and rename
50  • ALTER TABLE credit_card
51      DROP COLUMN expiring_date
52      ;
53  • ALTER TABLE credit_card
54      RENAME COLUMN expiring_date_temp TO expiring_date
55      ;
56

```

100% 2:55

Action Output

	Time	Action	Response	Duration / Fetch Time
✓ 1	22:03:28	ALTER TABLE credit_card DROP COL...	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.029 sec
✓ 2	22:03:43	ALTER TABLE credit_card RENAME C...	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.0063 sec

Després de crear la taula serà necessari que ingressis la informació del document denominat "dades_introduir_credit". Recorda mostrar el diagrama i realitzar una breu descripció d'aquest.

d.- Table relationships

The screenshot shows a database management interface. On the left, a tree view displays the database structure: transaction (Columns, Indexes, Foreign Keys, Triggers), Views, Stored Procedures, and Functions. The 'Foreign Keys' section is expanded, showing 'FK_transaction_creditcard' and 'transaction_ibfk_1'. The main area displays SQL code:

```

57  -- Construct the relationship transaction-credit-card
58  • ALTER TABLE `transaction`
59  ADD CONSTRAINT FK_transaction_creditcard
60  FOREIGN KEY (credit_card_id) REFERENCES credit_card(id)
61  ;

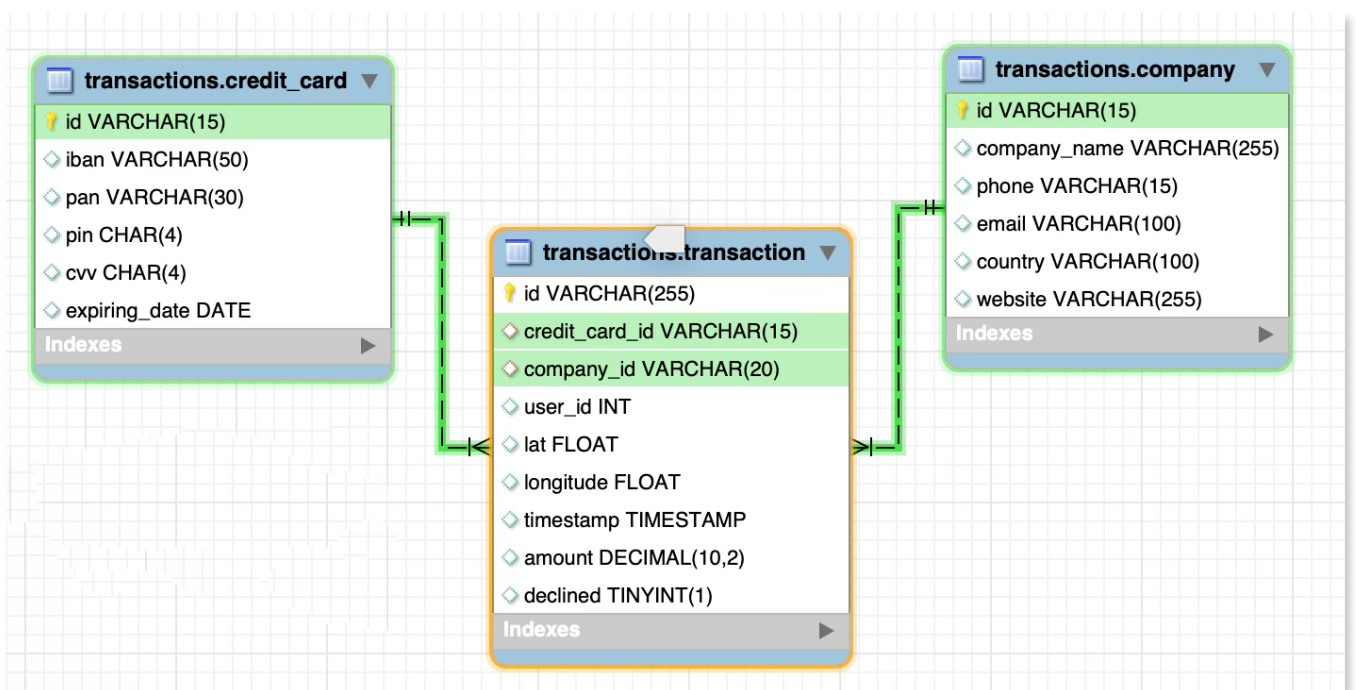
```

Below the code, the 'Action Output' tab shows the execution results:

	Time	Action	Response
1	14:20:30	ALTER TABLE `transaction` ADD CONSTRAINT FK_trans...	100000 row(s) affected Records: 100000 Duplicates: 0 War

We have the 'transaction' table as the fact table, while the other two tables act as dimension tables. At this stage, the database follows a **star schema dimensional structure**.

We can clearly identify that 'transaction' has a Primary Key that is the 'id' of each transaction and also 3 Foreign Key that relate it or will relate it to a cardinality of "N" many in 'transaction' to one in the dimension tables.



- Exercise 2

El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938. La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.

CcS-9580	XX781258889851950806677358	5541182364498931	9273	737	2029-03-27	
CcS-9581	XX915670516405388124398147	2624305470167630	4336	926	2025-06-29	
CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	2022-10-30	
CcU-2945	DO26854763748537475216568689	5142423821948828	9080	887	2023-08-24	
CcU-2952	BG45IVQL52710525608255	4556 453 55 5287	4598	438	2021-06-29	
CcU-2959	CR7242477244335841535	372461377349375	3583	667	2023-02-24	

```

70 • UPDATE credit_card
71 SET iban = 'TR323456312213576817699999'
72 WHERE id = 'CcU-2938'
73 ;
74

```

100%

2:73

Action Output

	Time	Action	Response
✓ 1	10:14:00	UPDATE credit_card SET iban = 'TR323456312213576817699999' WHERE id = 'CcU-2938'	1 row(s) affected Rov

CcS-9580	XX781258889851950806677358	5541182364498931	9273	737	2029-03-27	
CcS-9581	XX915670516405388124398147	2624305470167630	4336	926	2025-06-29	
CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30	
CcU-2945	DO26854763748537475216568689	5142423821948828	9080	887	2023-08-24	
CcU-2952	BG45IVQL52710525608255	4556 453 55 5287	4598	438	2021-06-29	

- Exercise 3

En la taula "transaction" ingressa una nova transacció amb la següent informació:

Id	108B1D1D-5B23-A76C-55EF-C568E49A99DD
credit_card_id	CcU-9999
company_id	b-9999
user_id	9999
lat	829.999
longitude	-117.999
amount	111.11
declined	0

I want to run the query at once, inserting the values on the PK of the dimensional tables if they are not there, to check this we use the clause "ON DUPLICATE KEY UPDATED". I will use START TRANSACTION - COMMIT that for me it looks a very interesting way

```

93 • START TRANSACTION;
94
95 • INSERT INTO credit_card (id)
96   VALUES ('CcU-9999')
97   ON DUPLICATE KEY UPDATE id = id;
98
99 • INSERT INTO company (id)
100  VALUES ('b-9999')
101  ON DUPLICATE KEY UPDATE id = id;
102
103 • INSERT INTO `transaction` (
104   id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
105   VALUES (
106     '108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', 9999, 829.999, -117.999, 111.11, 0);
107
108 • COMMIT;

```

100% 4:88

Action Output

	Time	Action	Response
✓ 1	14:26:39	START TRANSACTION	0 row(s) affected
✓ 2	14:26:39	INSERT INTO credit_card (id) VALUES ('CcU-9999') ON...	1 row(s) affected
✓ 3	14:26:39	INSERT INTO company (id) VALUES ('b-9999') ON DUPL...	1 row(s) affected
✓ 4	14:26:39	INSERT INTO `transaction` (id, credit_card_id, compan...	1 row(s) affected
✓ 5	14:26:39	COMMIT	0 row(s) affected

- Exercise 4

Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recordar mostrar el canvi realitzat.

The screenshot shows a database management interface. On the left, under 'Object Info', the table 'credit_card' is selected, showing its columns: id (varchar(15) PK), iban (varchar(50)), pin (char(4)), cvv (char(4)), and expiring_date (date). On the right, the SQL editor shows the following commands:

```

127 • ALTER TABLE credit_card
128 DROP COLUMN pan
129 ;
130

```

Below the editor, the 'Action Output' tab is active, displaying the execution results:

	Time	Action	Response
✓ 1	20:34:42	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Records:

Level 2

- Exercise 1

Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.

The screenshot shows a database management interface. The SQL editor contains the following commands:

```

128 • DELETE FROM `transaction`
129 WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'
130 ;

```

Below the editor, the 'Action Output' tab is active, displaying the execution results:

	Time	Action	Response
✓ 1	14:31:35	DELETE FROM `transaction` WHERE id = '000447FE-B6...	1 row(s) affected

- Exercise 2

La secció de màrqueting desitja tenir accés a informació específica per a realitzar anàlisi i estratègies efectives. S'ha sol·licitat crear una vista que proporcioni detalls clau sobre les companyies i les seves transaccions. Serà necessària que creïs una vista anomenada VistaMarketing que contingui la següent informació: Nom de la companyia. Telèfon de contacte. País de residència. Mitjana de compra realitzat per cada companyia. Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.

The screenshot shows a database management interface. On the left, a tree view shows the database structure with 'vistamarketing' selected under 'Views'. The 'Columns' section lists the fields: company_name (varchar(255)), phone (varchar(15)), country (varchar(100)), and average_purchase (decimal(11,2)). The main area displays the SQL code for creating the view:

```

139  -- To create a view, first we create a QUERY and then we use CREATE VIEW
140  -- and we ATTACHE THE QUERY to convert to permanent view. We use CREATE OR REPLACE
141  -- just in case the view already exists
142
143  • CREATE OR REPLACE VIEW vistamarketing AS
144  SELECT c.company_name,
145         c.phone,
146         c.country,
147         ROUND (AVG (t.amount),2) AS average_purchase
148  FROM company c
149  JOIN `transaction` t ON c.id = t.company_id
150  GROUP BY c.id, c.company_name, c.phone, c.country
151  ORDER BY AVG (t.amount) DESC
152  ;

```

The 'Action Output' section shows the execution result:

	Time	Action	Response
1	21:15:30	CREATE OR REPLACE VIEW vistamarketing AS SELECT c.company_name...	0 row(s) affected

When facing this exercise, I have had several doubts, the first in relation to the ORDER BY within the view. Actually according to MySQL 8.4 Reference Manual, ORDER BY is allowed inside the view but it will be ignored if there is an ORDER BY in the presentation of the view. The other doubt is in reference that in reality the exercise is asking for this ORDER BY out, but in that case if we do out an ORDER BY 'average_purchase' as it has round we lose precision.

Table resulting from running the view 'vistamarketing'

The screenshot shows the 'Result Grid' of the 'vistamarketing' view. The table has four columns: company_name, phone, country, and average_purchase. The data is as follows:

company_name	phone	country	average_purchase
Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.87
Pretium Neque Corp.	07 77 48 55 28	Australia	276.16
Urna Convallis Associates	06 01 24 77 04	United States	274.24

The 'Action Output' section shows the execution result:

	Time	Action	Response
1	13:33:34	SELECT * FROM transactions.vistamarketing	101 row(s) returned

- Exercise 3

Filtra la vista VistaMarketing per a mostrar només les companyies que tenen el seu país de residència en "Germany"

```

174 • SELECT *
175 FROM vistamarketing
176 WHERE country = 'Germany'
177 ;

```

100% 2:177

Result Grid



Filter Rows:

Search

Export:



	company_name	phone	country	average_purchase	
	Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.87	
	Nunc Interdum Incorporated	05 18 15 48 13	Germany	259.32	
	Convallis In Incorporated	06 66 57 29 50	Germany	257.75	
	Ac Industries	09 34 65 40 60	Germany	255.15	
	Rutrum Non Inc.	02 66 31 61 09	Germany	255.14	
	Auctor Mauris Corp.	05 62 87 14 41	Germany	254.77	
	Augue Foundation	06 88 43 15 63	Germany	253.51	
	Aliquam PC	01 45 73 52 16	Germany	253.14	

vistamarketing 6

Action Output



	Time	Action	Response
✓ 1	21:37:43	SELECT * FROM vistamarketing WHERE country = 'Germany'	8 row(s) returned

Level 3

- Exercise 1

La setmana vinent tindràs una nova reunió amb els gerents de màrqueting. Un company del teu equip va realitzar modificacions en la base de dades, però no recorda com les va realitzar. Et demana que l'ajudis a deixar els comandos executats per a obtenir el següent diagrama:

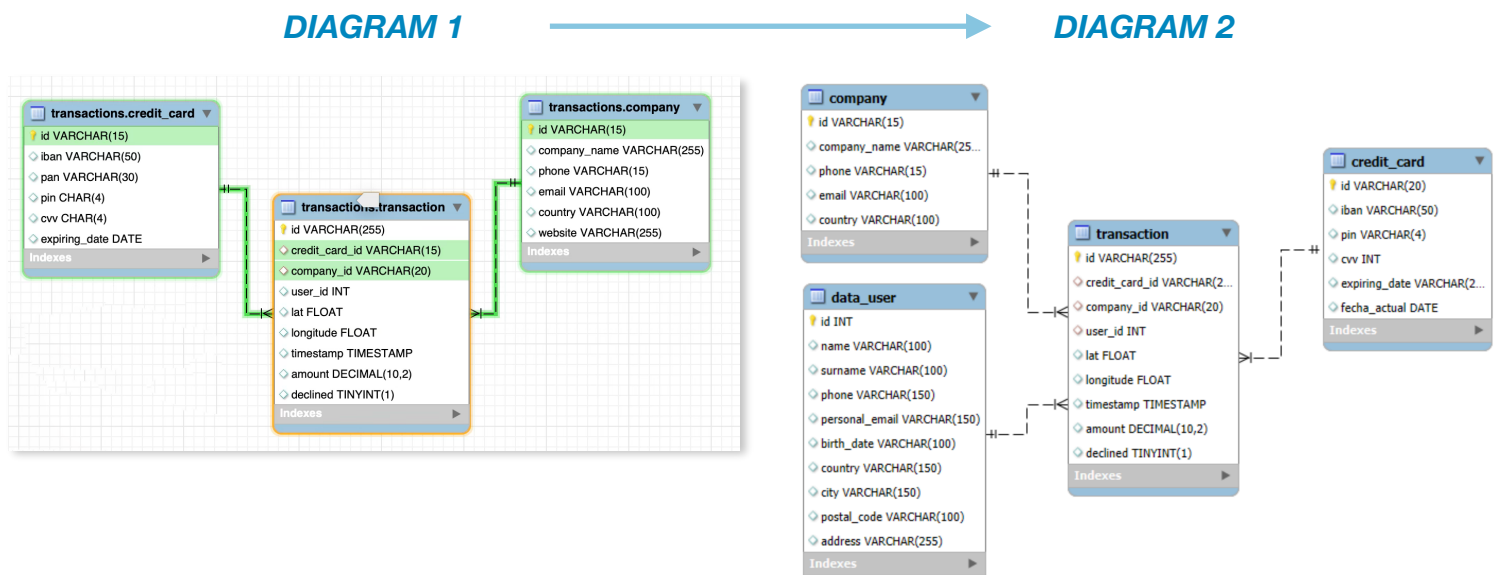
Recordatori:

En aquesta activitat, és necessari que descriguis el "pas a pas" de les tasques realitzades. És important realitzar descripcions senzilles, simples i fàcils de comprendre. Per a realitzar aquesta activitat hauràs de treballar amb els arxius denominats "estructura_dades_user" i "dades_introduir_user"

Recorda continuar treballant sobre el model i les taules amb les quals ja has treballat fins ara.

Objective:

Execute commands to obtain diagram 2 from diagram 1. Describe the transformation process in detail.



Step 1:

- Open in MySQL Workbench or the appropriate IDE the file: 'estructura datos user.sql'.
- Execute the script to create table 'user'.

Table: **user**

Columns:

Column	Type
id	char(10) PK
name	varchar(100)
surname	varchar(100)
phone	varchar(150)
email	varchar(150)
birth_date	varchar(100)
country	varchar(150)
city	varchar(150)
postal_code	varchar(100)
address	varchar(255)

```

8 • CREATE TABLE IF NOT EXISTS user (
9     id CHAR(10) PRIMARY KEY,
10    name VARCHAR(100),
11    surname VARCHAR(100),
12    phone VARCHAR(150),
13    email VARCHAR(150),
14    birth_date VARCHAR(100),
15    country VARCHAR(150),
16    city VARCHAR(150),
17    postal_code VARCHAR(100),
18    address VARCHAR(255)
19 );
20
  
```

Action Output

	Time	Action	Response
1	17:55:10	CREATE TABLE IF NOT EXISTS user (id CHAR(10) PRIM...	0 row(s) affected

- Open in your IDE the file with the data for 'user' table: 'datos introducir sprint3 user.sql'.
- Execute the script to insert the data in the table 'user'.

20 • INSERT INTO user (id, name, surname, phone, email, birth_date, country, city, postal_code, address) VALUES

Action Output

	Time	Action	Response
4998	18:00:45	INSERT INTO user (id, name, surname, phone, email, birt...	1 row(s) affected
4999	18:00:45	INSERT INTO user (id, name, surname, phone, email, birt...	1 row(s) affected
5000	18:00:45	INSERT INTO user (id, name, surname, phone, email, birt...	1 row(s) affected

Result Grid

id	name	surname	phone	email	birth_date	country	city	postal_code	address
10	Robert	Mccarthy	(324) 746-6771	fermentum@protonmail.com	Apr 30, 1984	United States	San Jose	95101	P.O. Box 77
100	Melodie	Mclean	1-677-221-7152	risus.varius@google.ca	Sep 15, 1989	United States	San Jose	95101	Ap #644-849
1000	Amelia	Obulyba	+49-958-9936	amkia.obulyba@example.com	May 17, 1970	Germany	Stuttgart	70173	215 Obulyba

Action Output

	Time	Action	Response	Duration
1	18:02:40	SELECT * FROM transactions.user	5000 row(s) returned	0.0013 se

Step 2: Rename table 'user' → 'data_user'

```

22 • ALTER TABLE `user`
23     RENAME TO data_user
24     ;

```

100% 1:18

Action Output

	Time	Action	Response
✓ 1	18:28:24	ALTER TABLE `user` RENAME TO data_user	0 row(s) affected

Step 3: Change column type 'id' from table 'data_user' to INT

```

29 • ALTER TABLE data_user
30     MODIFY COLUMN id INT
31     ;

```

100% 2:31

Action Output

	Time	Action	Response
✓ 1	18:40:58	ALTER TABLE data_user MODIFY COLUMN id INT	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

Step 4: Change column name 'email' from table 'data_user' to 'personal_email'

```

35 • ALTER TABLE data_user
36     RENAME COLUMN email TO personal_email
37     ;

```

100% 2:37

Action Output

	Time	Action	Response
✓ 1	18:50:57	ALTER TABLE data_user RENAME COLUMN email TO per...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

Step 5: Add row in the table 'data_user' with 'id' (9999) and the rest of the fields in blank. Necessary step to be able to create the FK constraint

```

43 • INSERT INTO data_user (id)
44     VALUES (9999)
45     ON DUPLICATE KEY UPDATE id = id
46     ;

```

100% 2:31

Action Output

	Time	Action	Response	Durat
✓ 2	19:38:47	INSERT INTO data_user (id) VALUES (9999) ON DU...	1 row(s) affected	0.042

Step 6: Create de FOREIGN KEY constraint between table 'transaction' and 'data_user'

```

219 • ALTER TABLE `transaction`
220 ADD CONSTRAINT FK_transaction_datauser
221 FOREIGN KEY (user_id) REFERENCES data_user(id)
222 ;

```

100% 44:49

Action Output

	Time	Action	Response
✓ 1	19:44:03	ALTER TABLE transaction ADD CONSTRAINT FK_tra...	100000 row(s) affected Records: 100000 Duplic

Step 7: Delete the 'website' column from the 'company' table

```

58 • ALTER TABLE company
59 DROP COLUMN website
60 ;

```

100% 2:60

Action Output

	Time	Action	Response
✓ 1	19:54:28	ALTER TABLE company DROP COLUMN website	0 row(s) affected Records: 0 Duplicates: 0 Warning

Step 8: Change the length of the 'credit_card_id' column of the 'transaction' table

```

65 • ALTER TABLE `transaction`
66 MODIFY COLUMN credit_card_id VARCHAR(20)
67 ;

```

100% 2:67

Action Output

	Time	Action	Response
✓ 1	20:28:24	ALTER TABLE `transaction` MODIFY COLUMN credi...	0 row(s) affected Records: 0 Duplicates: 0 W

Step 9: On the table 'credit_card' —> increase column length 'id'

```
72 • ALTER TABLE credit_card
73   MODIFY COLUMN id VARCHAR(20)
74   ;
```

100% 35:70

Action Output

	Time	Action	Response
✓ 1	20:36:30	ALTER TABLE credit_card MODIFY COLUMN id VAR...	0 row(s) affected Records: 0 Duplicates: 0 Warning

Step 10: On the table 'credit_card' —> change 'pin' data type

```
79 • ALTER TABLE credit_card
80   MODIFY COLUMN pin VARCHAR(4)
81   ;
```

100% 1:75

Action Output

	Time	Action	Response
✓ 1	20:41:42	ALTER TABLE credit_card MODIFY COLUMN pin VA...	5001 row(s) affected Records: 5001 Duplicates: 0...

Step 11: On the table 'credit_card' —> change 'cvv' data type

```
86 • ALTER TABLE credit_card
87   MODIFY COLUMN cvv INT
88   ;
```

100% 24:79

Action Output

	Time	Action	Response
✓ 1	20:45:13	ALTER TABLE credit_card MODIFY COLUMN cvv INT	5001 row(s) affected Records: 5001 Duplicates: 0...

Step 12: On the table 'credit_card' —> change 'expiring_date' data type

```
94 • ALTER TABLE credit_card
95   MODIFY COLUMN expiring_date VARCHAR(20)
96   ;
```

100% 3:92

Action Output

	Time	Action	Response
✓ 1	20:49:39	ALTER TABLE credit_card MODIFY COLUMN expirin...	5001 row(s) affected Records: 5001 Duplicate

Step 13: On the table 'credit_card' → create new column 'fecha_actual'

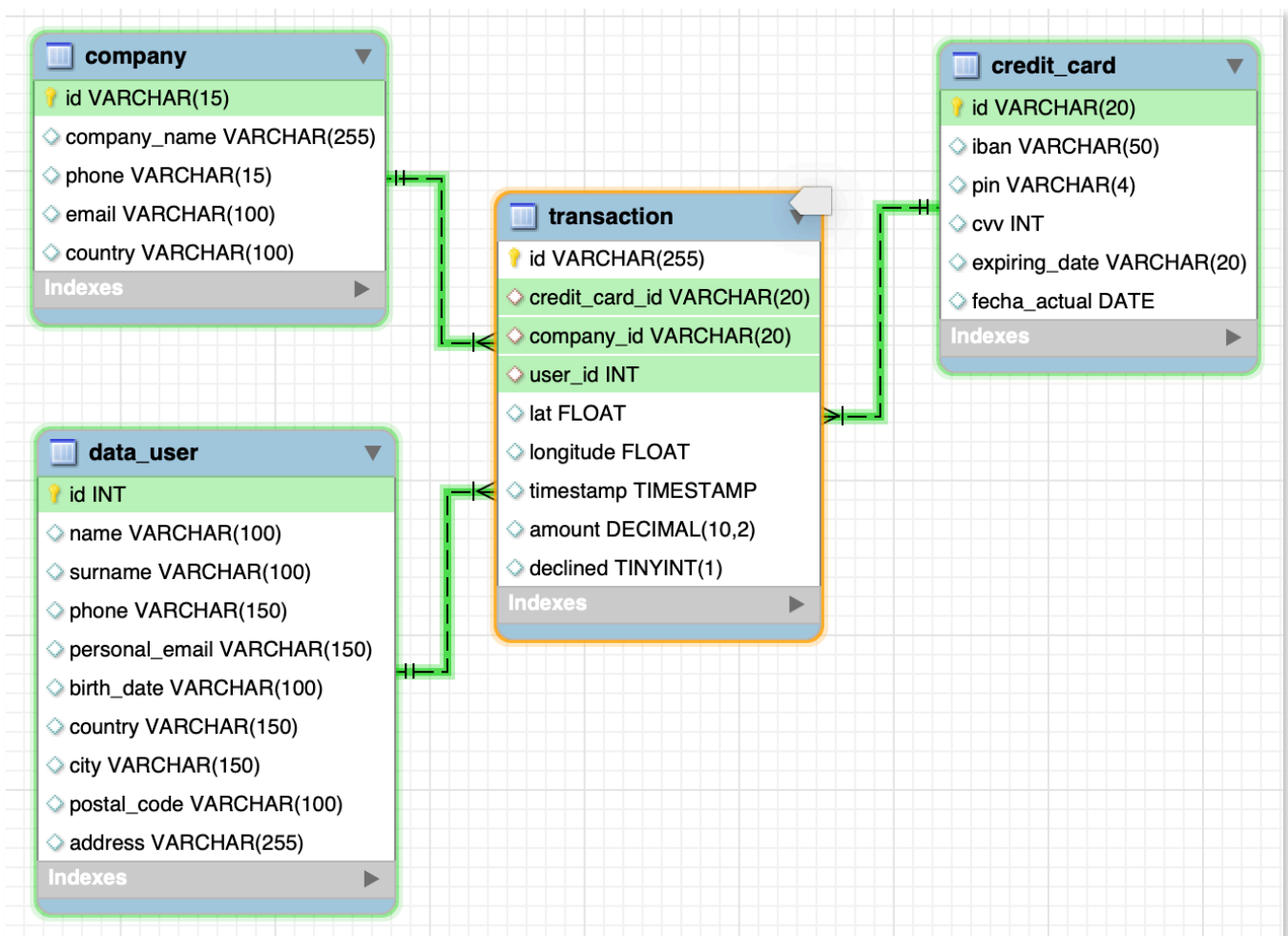
```
100 • ALTER TABLE credit_card
101     ADD fecha_actual DATE
102     ;
```

100% 1:96

Action Output

	Time	Action	Response
✓ 1	20:54:22	ALTER TABLE credit_card ADD fecha_actual DATE	0 row(s) affected Records: 0 Duplicates: 0 Warnir

Below you can see the final diagram result of applying all these SQL scripts on the transaction database.



As a final note of exercise 1, I want to explain that I have made the DDL (Data Definition Language - structure modification) and DML (Data Manipulation Language - data modifications) of this exercise step by step, because the statement specified it.

But it would be much more efficient, grouping the DDL by tables with an ALTER per table and the DMLs also separately.

In this case, we cannot use TCL (Transaction Control Language) START TRANSACTION - COMMIT; since DDL makes COMMIT.

Below there is an example of how the exercise would look focusing on efficiency:

```
287  -- USER
288  ● ALTER TABLE `user`
289    RENAME TO data_user,
290    MODIFY COLUMN id INT,
291    RENAME COLUMN email TO personal_email
292  ;
293  ● INSERT INTO data_user (id)
294    VALUES (9999)
295    ON DUPLICATE KEY UPDATE id = id
296  ;
297  -- TRANSACTION
298  ● ALTER TABLE `transaction`
299    MODIFY COLUMN credit_card_id VARCHAR(20),
300    ADD CONSTRAINT FK_transaction_datauser
301    FOREIGN KEY (user_id) REFERENCES data_user(id)
302  ;
303  -- COMPANY
304  ● ALTER TABLE company
305    DROP COLUMN website
306  ;
307  -- CREDIT_CARD
308  ● ALTER TABLE credit_card
309    MODIFY COLUMN id VARCHAR(20),
310    MODIFY COLUMN pin VARCHAR(4),
311    MODIFY COLUMN cvv INT,
312    MODIFY COLUMN expiring_date VARCHAR(20),
313    ADD COLUMN fecha_actual DATE
314  ;
```

- Exercise 2

L'empresa també us demana crear una vista anomenada "InformeTecnico" que contingui la següent informació:

ID de la transacció

Nom de l'usuari/ària

Cognom de l'usuari/ària

IBAN de la targeta de crèdit usada.

Nom de la companyia de la transacció realitzada.

Assegureu-vos d'incloure informació rellevant de les taules que coneixereu i utilitzeu àlies per canviar de nom columnes segons calgui.

Mostra els resultats de la vista, ordena els resultats de forma descendent en funció de la variable ID de transacció.

Apart from the columns requested by the exercise, I have added several columns that seemed relevant to me or that could give us relevant information:

- amount: allows us to know how relevant a transaction is.
- birthday: it would allow to find out the age of the customers
- company_country and user_country: would allow us to establish if there is any relationship between the place of residence of a user and if they buy from companies of the same country.
- card_expiration_date and operation_declined: It would allow to check if there is any relationship between cancellations and the expiration dates of credit cards.

On the next page, you'll find the script for the creation of the view requested.

```

330 • CREATE OR REPLACE VIEW informetecnico AS
331 SELECT t.id AS transactionID,
332        cc.iban,
333        DATE(t.timestamp) AS transaction_date,
334        t.amount,
335        du.name AS user_name,
336        du.surname AS user_surname,
337        STR_TO_DATE(du.birth_date, '%b %e, %Y') AS birthday,
338        du.city AS user_city,
339        du.country AS user_country,
340        c.company_name,
341        c.country AS company_country,
342        cc.expiring_date AS card_expiration_date,
343        t.declined AS operation_declined
344 FROM `transaction` t
345 JOIN data_user du ON t.user_id = du.id
346 JOIN company c ON t.company_id = c.id
347 JOIN credit_card cc ON t.credit_card_id = cc.id
348 ;

```

100% 2:353

Action Output

	Time	Action	Response
1	21:42:29	CREATE OR REPLACE VIEW informetecnico AS SELECT t.id AS transactio...	0 row(s) affected

In this exercise, a problem similar to the exercise Level 2 - Exercise 2 is presented, but here there is no doubt that the ORDER BY clause is applied in the presentation of the view rather than within the view itself: “Mostra els resultats de la vista, ordena els resultats de forma descendent en funció de la variable ID de transacció.”

```

350 • SELECT *
351 FROM informetecnico i
352 ORDER BY i.transactionID DESC
353 ;

```

100% 50:342

transactionID	iban	transaction_date	amount	user_name	user_surname	birthday	user_city	user_country	company_name	company_country	card_expiration_date	operation_declined
FFFC76D-ECF0-4985-A2D0-B2A7...	XX636251701647892036676034	2023-06-17	148.91	Dlried	Vlqcdjl	1969-12-18	The Hague	Netherlands	Amet Nulla Donec Corporation	Italy	2028-01-28	0
FFFC9E8D-27C7-4ADE-98F2-7533E...	XX162677143304223631437567	2019-10-30	234.22	Securp	Faotvqly	1957-11-13	Stockholm	Sweden	Nunc Interdum Incorporated	Germany	2027-10-28	0
FFFB270D-F53A-4D5D-9666-E5307...	XX395114267082019952567052	2024-05-13	349.13	Ggzjpa	Uirzjulh	1988-12-24	Funchal	Portugal	Viverra Donec Foundation	United Kingdom	2028-04-28	0
FFF9E3CE-234E-408C-A8EF-F9CA...	XX8845462156537570367941	2022-12-17	247.39	Yshimq	Zpsjsleed	1965-05-20	Berlin	Germany	Convallis In Incorporated	Germany	2027-03-26	0
FFF9E178-6CD2-4DF9-99B0-49AE0...	XX321405515711654384711481	2023-02-08	438.13	Jevepx	Xwcwzwnm	1995-11-25	Rotterdam	Netherlands	Mus Aenean Eget Foundation	Sweden	2028-02-27	0
FFFB67C9-17B5-4B1F-AFD9-F8023...	XX278446342932680979729426	2015-04-12	448.63	Fqlngd	Lvhfyxi	1967-05-04	Birmingham	United Kingdom	Cras Vehicula Aliquet Industries	Netherlands	2025-10-26	0
FFF7042D-18C6-4DDD-823C-4D90...	XX405009272572550082027209	2017-07-20	163.35	Njoraa	Egsqquii	1956-09-13	Amadora	Portugal	Placerat LLP	Netherlands	2027-03-27	0
FFF660D4-4244-47F6-9210-ESD1D...	XX63376659736627454015125	2017-05-18	288.76	Lopzaj	llyrylay	1978-04-09	Dallas	United States	Pede Cum Ltd	Norway	2029-11-26	0

informetecnico 1

Action Output

	Time	Action	Response
1	21:47:44	SELECT * FROM informetecnico i ORDER BY i.transactionID DESC	100000 row(s) returned